```
import os
from PySide6.QtWidgets import (QDockWidget,
                               QWidget,
                               QVBoxLayout,
                               QHBoxLayout,
                               QPushButton,
                               QLineEdit,
                               QLabel,
                               QRadioButton,
                               QButtonGroup,
                               QGridLayout,
                               )
from PySide6.QtCore import QEvent, Qt, Signal, Slot
from PySide6.QtGui import QTextCursor

class LatexIndexWindow(QDockWidget):
    # Broadcast structural updates to the sidebar tree view
    indexInserted = Signal(list, dict)

    # ARCHITECTURAL SIGNALS: Hand off file operations to the main window controller
    saveRequested = Signal(object, list)  # Emits (editor_widget, result_holder_list)
    syncRequested = Signal(object, str)   # Emits (editor_widget, file_path_string)
    # Intercepted by MainWindow to request a guaranteed unique incremental identifier number
    nextIdRequested = Signal(list) # Emits a mutable list containing [next_id_int]

    def __init__(self, title="LaTeX Index Entry", parent=None, tab_widget=None):
        super().__init__(title, parent)

        self.tab_widget = tab_widget

        self.setObjectName("LatexIndexWindow")
        self.setFeatures(QDockWidget.NoDockWidgetFeatures)
        self.setAllowedAreas(Qt.BottomDockWidgetArea)

        # Initialize focus tracking variable to prevent AttributeError on startup
        self.last_focused_field = None

        self.container = QWidget()
        self.layout = QVBoxLayout(self.container)
        self.layout.setContentsMargins(5, 5, 5, 5)

        # --- Entry Input Fields ---
        self.input_layout = QGridLayout()
        self.main_label = QLabel("Main:")
        self.main_entry = QLineEdit(placeholderText="Main Entry")
        self.main_entry.returnPressed.connect(self.reveal_sub1)

        self.sub1_label = QLabel("Subhead 1:")
        self.sub1_entry = QLineEdit(placeholderText="Subheading 1")
        self.sub1_entry.returnPressed.connect(self.reveal_sub2)

        self.sub2_label = QLabel("Subhead 2:")
        self.sub2_entry = QLineEdit(placeholderText="Subheading 2")

        # Hide subheaders initially
        for w in [self.sub1_label, self.sub1_entry, self.sub2_label, self.sub2_entry]:
            w.hide()

        self.input_layout.addWidget(self.main_label, 0, 0)
        self.input_layout.addWidget(self.main_entry, 0, 1)
        self.input_layout.addWidget(self.sub1_label, 1, 0)
        self.input_layout.addWidget(self.sub1_entry, 1, 1)
        self.input_layout.addWidget(self.sub2_label, 2, 0)
        self.input_layout.addWidget(self.sub2_entry, 2, 1)
        self.layout.addLayout(self.input_layout)

        # --- Button Bar (Entry Formatting & Page Style) ---
        self.bar_layout = QHBoxLayout()

        # Entry Text Formatting (Toggle buttons)
        self.bold_entry = QPushButton("B")
        self.bold_entry.setCheckable(True)
        self.bold_entry.setFixedWidth(30)
        self.bold_entry.setStyleSheet("""
            QPushButton {
                font-family: "Verdana", sans-serif;
                font-size: 14px;
                font-weight: bold;
            }
            QPushButton:checked {
                background-color: lightblue;
            }
        """)
        self.bold_entry.setFocusPolicy(Qt.FocusPolicy.NoFocus)
        self.bold_entry.setToolTip("Bold the text in the entry field")
        self.bold_entry.clicked.connect(lambda: self.format_selected_text("textbf"))
```

```
        self.ital_entry = QPushButton("I")
        self.ital_entry.setCheckable(True)
        self.ital_entry.setFixedWidth(30)
        self.ital_entry.setStyleSheet("""
            QPushButton {
                font-family: "Verdana", sans-serif;
                font-size: 14px;
                font-style: italic;
            }
            QPushButton:checked {
                background-color: lightblue;
            }
        """)
        self.ital_entry.setFocusPolicy(Qt.FocusPolicy.NoFocus)
        self.ital_entry.setToolTip("Italicize the text in the entry field")
        self.ital_entry.clicked.connect(lambda: self.format_selected_text("textit"))

        self.format_group = QButtonGroup(self)
        for btn in [self.bold_entry, self.ital_entry]:
            self.format_group.addButton(btn)

        # Page Reference Styles (Radio buttons)
        self.none_ref = QRadioButton("Plain")
        self.bold_ref = QRadioButton("Bold Page")
        self.italic_ref = QRadioButton("Italic Page")
        self.none_ref.setChecked(True)

        self.style_group = QButtonGroup(self)
        for btn in [self.none_ref, self.bold_ref, self.italic_ref]:
            self.style_group.addButton(btn)

        # Install event filters to track which field has focus
        for field in [self.main_entry, self.sub1_entry, self.sub2_entry]:
            field.installEventFilter(self)

        self.insert_btn = QPushButton("Insert Index Tag")
        self.insert_btn.setShortcut("Ctrl+K")
        self.insert_btn.clicked.connect(self.handle_insert)

        # Add all to bar
        self.bar_layout.addWidget(QLabel("Text Style:"))
        self.bar_layout.addWidget(self.bold_entry)
        self.bar_layout.addWidget(self.ital_entry)
        self.bar_layout.addSpacing(20)
        self.bar_layout.addWidget(QLabel("Page Ref:"))
        self.bar_layout.addWidget(self.none_ref)
        self.bar_layout.addWidget(self.bold_ref)
        self.bar_layout.addWidget(self.italic_ref)
        self.bar_layout.addStretch()
        self.bar_layout.addWidget(self.insert_btn)

        self.layout.addLayout(self.bar_layout)
        self.setWidget(self.container)

    def reveal_sub1(self):
        if self.main_entry.text().strip():
            self.sub1_label.show()
            self.sub1_entry.show()
            self.sub1_entry.setFocus()

    def reveal_sub2(self):
        if self.sub1_entry.text().strip():
            self.sub2_label.show()
            self.sub2_entry.show()
            self.sub2_entry.setFocus()

    def eventFilter(self, obj, event):
        """Track which QLineEdit has focus."""
        if event.type() == QEvent.Type.FocusIn and isinstance(obj, QLineEdit):
            self.last_focused_field = obj
        return super().eventFilter(obj, event)

    def format_selected_text(self, command):
        # Wraps ONLY the selected text in the active field with \command{...}
        field = self.last_focused_field
        if not field or not field.hasSelectedText():
            return

        start = field.selectionStart()
        length = len(field.selectedText())
        full_text = field.text()

        before = full_text[:start]
        selection = field.selectedText()
        after = full_text[start + length:]
```

```
    new_text = f"{before}\\{command}{{{selection}}}{after}"
    field.setText(new_text)
    field.setFocus()

def insert_latex(self, chain, pg_style):
    editor = self.tab_widget.currentWidget()
    if not editor:
        return

    cursor = editor.textCursor()

    # Check if the user has selected a range of text in the editor canvas
    if cursor.hasSelection():
        selected_text = cursor.selectedText()

        # Combine any explicit page styling with the structural range operators
        start_format = f"|{pg_style}|(" if pg_style else "|("
        end_format = f"|{pg_style}|)" if pg_style else "|)"

        # Assemble the wrapping macros
        start_tag = f"\\index{{{chain}{start_format}}}"
        end_tag = f"\\index{{{chain}{end_format}}}"

        # Construct the surrounded block layout text stream
        wrapped_text = f"{start_tag}{selected_text}{end_tag}"

        # Overwrite the highlight block cleanly with the wrapped payload
        cursor.insertText(wrapped_text)
    else:
        # Fallback behavior: If no text is selected, drop a standard local point tag
        if pg_style:
            macro_tag = f"\\index{{{chain}|{pg_style}}}"
        else:
            macro_tag = f"\\index{{{chain}}}"
        cursor.insertText(macro_tag)

    editor.setTextCursor(cursor)

def reset_ui(self):
    """Clears text fields and resets focus back to the top entry row."""
    self.main_entry.clear()
    self.sub1_entry.clear()
    self.sub2_entry.clear()

    for w in [self.sub1_label, self.sub1_entry, self.sub2_label, self.sub2_entry]:
        w.hide()

    self.none_ref.setChecked(True)
    if self.format_group.checkedButton():
        self.format_group.setExclusive(False)
        self.format_group.checkedButton().setChecked(False)
        self.format_group.setExclusive(True)

    self.main_entry.setFocus()

def handle_insert(self):
    """Validates entry states, coordinates file tracking, and applies tag macros."""
    editor = self.tab_widget.currentWidget()
    if not editor:
        return

    path = getattr(editor, 'file_path', 'Untitled')
    if path == 'Untitled':
        save_result = []
        self.saveRequested.emit(editor, save_result)
        if not save_result or not save_result[0]:
            print("DEBUG: Index tag insertion aborted because file was not saved.")
            return

        path = getattr(editor, 'file_path', 'Untitled')
        if path == 'Untitled':
            return

    def process_field(field):
        val = field.text().strip()
        if not val:
            return None
        if field == self.last_focused_field:
            styled = val
            if self.bold_entry.isChecked(): styled = f"\\textbf{{{styled}}}"
            if self.ital_entry.isChecked(): styled = f"\\textit{{{styled}}}"
            if styled != val: return f"{val}@{styled}"
        return val

    m = process_field(self.main_entry)
    if not m:
        return
```

```
        s1 = process_field(self.sub1_entry)
        s2 = process_field(self.sub2_entry)

        # self.window() bypasses all docks/splitters and finds the true top-level QMainWindow
        main_win = self.window()

        # Fallback check: if somehow self.window() is self, try parent() tree traversal
        if not hasattr(main_win, 'get_and_increment_id'):
            parent_widget = self.parent()
            while parent_widget is not None:
                if hasattr(parent_widget, 'get_and_increment_id'):
                    main_win = parent_widget
                    break
                parent_widget = parent_widget.parent()

        # Execute transaction if the verified window object is located
        if main_win and hasattr(main_win, 'get_and_increment_id'):
            assigned_idn = main_win.get_and_increment_id()
        else:
            print("ERROR: Verifiable MainWindow state tracking engine could not be resolved.")
            return

        cursor = editor.textCursor()
        line = cursor.blockNumber() + 1
        col = cursor.columnNumber() + 1

        # Extract the line and column markers from the START of the selection range
        # to ensure navigation tracks perfectly even if the selection spans multiple lines.
        if cursor.hasSelection():
            # Create a copy cursor anchored exactly at the beginning of the highlighted range
            start_cursor = editor.textCursor()
            start_cursor.setPosition(cursor.selectionStart())
            line = start_cursor.blockNumber() + 1
            col = start_cursor.columnNumber() + 1
        else:
            line = cursor.blockNumber() + 1
            col = cursor.columnNumber() + 1

        chain = "!".join([p for p in [m, s1, s2] if p])
        pg_style = "bold" if self.bold_ref.isChecked() else "italic" if self.italic_ref.isChecked() else None


        # Apply structural macro edits onto the open editor document layer
        self.insert_latex(chain, pg_style)

        # Trigger disk flushing pipeline via syncRequested signal hook
        self.syncRequested.emit(editor, path)

        # Build unique identification schema dictionary mapping requirements perfectly
        uid_dict = {
            "id": assigned_idn,
            "path": os.path.normpath(path),
            "line": int(line),
            "col": int(col)
        }

        # Inform index tree sidebar about the newly established node elements
        parts_list = [p for p in [m, s1, s2] if p]
        self.indexInserted.emit(parts_list, uid_dict)

        self.reset_ui()

# from PySide6.QtWidgets import (QDockWidget,
#                                QWidget,
#                                QVBoxLayout,
#                                QHBoxLayout,
#                                QPushButton,
#                                QLineEdit,
#                                QLabel,
#                                QRadioButton,
#                                QButtonGroup,
#                                QGridLayout,
#                                )
# from PySide6.QtCore import QEvent, Qt, Signal

# class LatexIndexWindow(QDockWidget):
#     # Define a signal that emits parts_with_style, file_path, line, column
#     indexInserted = Signal(list, str, int, int)

#     def __init__(self, title="LaTeX Index Entry", parent=None, tab_widget=None):
#         super().__init__(title, parent)
#         self.tab_widget = tab_widget
#         self.setObjectName("LatexIndexWindow")
#         self.setFeatures(QDockWidget.NoDockWidgetFeatures)
#         self.setAllowedAreas(Qt.BottomDockWidgetArea)
```

```
#         self.container = QWidget()
#         self.layout = QVBoxLayout(self.container)
#         self.layout.setContentsMargins(5, 5, 5, 5)

#         # --- Entry Input Fields ---
#         self.input_layout = QGridLayout()
#         self.main_label = QLabel("Main:")
#         self.main_entry = QLineEdit(placeholderText="Main Entry")
#         self.main_entry.returnPressed.connect(self.reveal_sub1)

#         self.sub1_label = QLabel("Subhead 1:")
#         self.sub1_entry = QLineEdit(placeholderText="Subheading 1")
#         self.sub1_entry.returnPressed.connect(self.reveal_sub2)

#         self.sub2_label = QLabel("Subhead 2:")
#         self.sub2_entry = QLineEdit(placeholderText="Subheading 2")

#         # Hide subheaders initially
#         for w in [self.sub1_label, self.sub1_entry, self.sub2_label, self.sub2_entry]:
#             w.hide()

#         self.input_layout.addWidget(self.main_label, 0, 0)
#         self.input_layout.addWidget(self.main_entry, 0, 1)
#         self.input_layout.addWidget(self.sub1_label, 1, 0)
#         self.input_layout.addWidget(self.sub1_entry, 1, 1)
#         self.input_layout.addWidget(self.sub2_label, 2, 0)
#         self.input_layout.addWidget(self.sub2_entry, 2, 1)
#         self.layout.addLayout(self.input_layout)

#         # --- Button Bar (Entry Formatting & Page Style) ---
#         self.bar_layout = QHBoxLayout()

#         # Entry Text Formatting (Toggle buttons)
#         self.bold_entry = QPushButton("B")
#         self.bold_entry.setCheckable(True)
#         self.bold_entry.setFixedWidth(30)
#         self.bold_entry.setStyleSheet("""
#             QPushButton {
#                 font-family: "Verdana", sans-serif;
#                 font-size: 14px;
#                 font-weight: bold;
#             }
#             QPushButton:checked {
#                 background-color: lightblue;
#             }
#         """)
#         self.bold_entry.setFocusPolicy(Qt.FocusPolicy.NoFocus)  # Prevent button from stealing focus
#         self.bold_entry.setToolTip("Bold the text in the entry field")
#         self.bold_entry.clicked.connect(lambda: self.format_selected_text("textbf"))

#         self.ital_entry = QPushButton("I")
#         self.ital_entry.setCheckable(True)
#         self.ital_entry.setFixedWidth(30)
#         self.ital_entry.setStyleSheet("""
#             QPushButton {
#                 font-family: "Verdana", sans-serif;
#                 font-size: 14px;
#                 font-style: italic;
#             }
#             QPushButton:checked {
#                 background-color: lightblue;
#             }
#         """)
#         self.ital_entry.setFocusPolicy(Qt.FocusPolicy.NoFocus)  # Prevent button from stealing focus
#         self.ital_entry.setToolTip("Italicize the text in the entry field")
#         self.ital_entry.clicked.connect(lambda: self.format_selected_text("textit"))

#         self.format_group = QButtonGroup(self)
#         for btn in [self.bold_entry, self.ital_entry]: self.format_group.addButton(btn)

#         # Page Reference Styles (Radio buttons)
#         self.none_ref = QRadioButton("Plain")
#         self.bold_ref = QRadioButton("Bold Page")
#         self.italic_ref = QRadioButton("Italic Page")
#         self.none_ref.setChecked(True)

#         self.style_group = QButtonGroup(self)
#         for btn in [self.none_ref, self.bold_ref, self.italic_ref]: self.style_group.addButton(btn)

#         # Install event filters to track which field has focus
#         for field in [self.main_entry, self.sub1_entry, self.sub2_entry]:
#             field.installEventFilter(self)

#         self.insert_btn = QPushButton("Insert Index Tag")
#         self.insert_btn.setShortcut("Ctrl+K")
#         self.insert_btn.clicked.connect(self.handle_insert)
```

```
#         # Add all to bar
#         self.bar_layout.addWidget(QLabel("Text Style:"))
#         self.bar_layout.addWidget(self.bold_entry)
#         self.bar_layout.addWidget(self.ital_entry)
#         self.bar_layout.addSpacing(20)
#         self.bar_layout.addWidget(QLabel("Page Ref:"))
#         self.bar_layout.addWidget(self.none_ref)
#         self.bar_layout.addWidget(self.bold_ref)
#         self.bar_layout.addWidget(self.italic_ref)
#         self.bar_layout.addStretch()
#         self.bar_layout.addWidget(self.insert_btn)

#         self.layout.addLayout(self.bar_layout)
#         self.setWidget(self.container)

#     def reveal_sub1(self):
#         if self.main_entry.text().strip():
#             self.sub1_label.show(); self.sub1_entry.show(); self.sub1_entry.setFocus()

#     def reveal_sub2(self):
#         if self.sub1_entry.text().strip():
#             self.sub2_label.show(); self.sub2_entry.show(); self.sub2_entry.setFocus()

#     def eventFilter(self, obj, event):
#         """Track which QLineEdit has focus."""
#         if event.type() == QEvent.Type.FocusIn and isinstance(obj, QLineEdit):
#             self.last_focused_field = obj
#         return super().eventFilter(obj, event)

#     def format_selected_text(self, command):
#         # Wraps ONLY the selected text in the active field with \command{...}
#         field = self.last_focused_field
#         if not field or not field.hasSelectedText():
#             return

#         # Get selection bounds
#         start = field.selectionStart()
#         length = len(field.selectedText())
#         full_text = field.text()

#         # Extract parts
#         before = full_text[:start]
#         selection = field.selectedText()
#         after = full_text[start + length:]

#         # Reconstruct with LaTeX tags
#         new_text = f"{before}\\{command}{{{selection}}}{after}"
#         field.setText(new_text)

#         # Set cursor back to field
#         field.setFocus()

#     def handle_insert(self):
#         editor = self.tab_widget.currentWidget()
#         if not editor:
#             return

#         # 1. Handle unsaved "Untitled" buffers
#         initial_path = getattr(editor, 'file_path', 'Untitled')
#         if initial_path == 'Untitled':
#             main_win = self.window()  # Safely fetches top-level MainWindow context

#             if hasattr(main_win, 'save_file_as'):
#                 # Invoke the native save dialog wrapper
#                 save_success = main_win.save_file_as(editor)
#                 if not save_success:
#                     print("DEBUG: Index tag insertion aborted because file was not saved.")
#                     return
#             else:
#                 print("ERROR: MainWindow is missing 'save_file_as' method.")
#                 return

#         # CRITICAL FIX: Re-fetch the real property directly from the editor instance object.
#         # Your save_file_as() method must update `editor.file_path = absolute_saved_path` internally.
#         path = getattr(editor, 'file_path', 'Untitled')
#         if path == 'Untitled':
#             print("ERROR: File was saved, but editor.file_path attribute was not updated by save_file_as().")
#             return

#         def process_field(field):
#             val = field.text().strip()
#             if not val: return None
#             if field == self.last_focused_field:
#                 styled = val
#                 if self.bold_entry.isChecked(): styled = f"\\textbf{{{styled}}}"
#                 if self.ital_entry.isChecked(): styled = f"\\textit{{{styled}}}"
#                 if styled != val: return f"{val}@{styled}"
```

```
#             return val

#         m = process_field(self.main_entry)
#         if not m: return

#         s1 = process_field(self.sub1_entry)
#         s2 = process_field(self.sub2_entry)

#         cursor = editor.textCursor()
#         line = cursor.blockNumber() + 1
#         col = cursor.columnNumber() + 1

#         chain = "!".join([p for p in [m, s1, s2] if p])
#         pg_style = "bold" if self.bold_ref.isChecked() else "italic" if self.italic_ref.isChecked() else None

#         # 2. Inject text macro string to editor layer memory layout
#         self.insert_latex(chain, pg_style)

#         # 3. Synchronize file system directly with session lifecycle framework
#         main_win = self.window()
#         if hasattr(main_win, 'save_tex_file_to_disk'):
#             main_win.save_tex_file_to_disk(editor, path)
#             main_win._tree_modified = True
#         else:
#             try:
#                 with open(path, 'w', encoding='utf-8') as f:
#                     f.write(editor.toPlainText())
#             except Exception as e:
#                 print(f"ERROR: Direct disk synchronization fallback failed: {e}")

#         # 4. Broadcast structural addition parameters to index tree model nodes
#         parts_with_style = [p for p in [m, s1, s2] if p]
#         self.indexInserted.emit(parts_with_style, path, line, col)

#         self.reset_ui()

#     def insert_latex(self, entry_chain, style):
#         editor = self.tab_widget.currentWidget()
#         if not editor or not hasattr(editor, 'textCursor'): return

#         cursor = editor.textCursor()
#         fmt = {"bold": "textbf", "italic": "textit"}.get(style)

#         if cursor.hasSelection():
#             start, end = cursor.selectionStart(), cursor.selectionEnd()
#             cursor.setPosition(end)
#             cursor.insertText(f"\\index{{{entry_chain}|)}}")
#             cursor.setPosition(start)
#             s_sfx = f"|({fmt}" if fmt else "|("
#             cursor.insertText(f"\\index{{{entry_chain}{s_sfx}}}")
#         else:
#             p_sfx = f"|{fmt}" if fmt else ""
#             cursor.insertText(f"\\index{{{entry_chain}{p_sfx}}}")

#     def reset_ui(self):
#         for f in [self.main_entry, self.sub1_entry, self.sub2_entry]: f.clear()
#         for w in [self.sub1_label, self.sub1_entry, self.sub2_label, self.sub2_entry]: w.hide()
#         self.bold_entry.setChecked(False)
#         self.ital_entry.setChecked(False)
#         self.none_ref.setChecked(True)
#         self.main_entry.setFocus()
```